

Lezione 2: Esercitazione Pratica

Collezioni, Interfacce Funzionali e Generics in Java

Michele Gentile

7 Maggio 2026

1 Sistema di Gestione Dispositivi Smart

1. Si definisca una classe **Dispositivo** i cui oggetti sono costruiti con un **codice univoco** (String), un **nome** (es. "Lampada Soggiorno") e il **consumo energetico** (double).
2. Si crei una classe **SmartHome** che contiene un insieme di dispositivi utilizzando un **HashSet**.
3. Implementare i metodi **equals** e **hashCode** nella classe *Dispositivo* affinché due oggetti siano considerati duplicati se hanno lo stesso codice univoco.
4. Fornire una rappresentazione testuale chiara tramite il metodo **toString**.
5. Si implementi la sottoclasse **SmartHomeOrdinata** che utilizzi un **TreeSet** per mantenere i dispositivi ordinati in base al **consumo energetico** e, a parità di consumo, in base al **nome**.

🔗 Obiettivo: Gestione Set e Ordinamento

L'esercizio mira a consolidare la comprensione del contratto tra **equals/hashCode** per le collezioni basate su hash e l'interfaccia **Comparable** per le collezioni ordinate.

💡 Tip: Coerenza tra equals e compareTo

Per evitare comportamenti inattesi nel **TreeSet**, assicuratevi che **compareTo** restituisca 0 se e solo se **equals** restituisce **true** (ove possibile) o gestite esplicitamente i criteri di spareggio.

2 Pipeline di Elaborazione Stringhe

Creare una classe **TextProcessor** i cui oggetti gestiscono una sequenza di trasformazioni testuali.

1. La classe deve memorizzare una lista di funzioni (`java.util.function.Function<String, String>`).

2. Implementare un metodo `aggiungiFase(Function<String, String> f)` per inserire nuove trasformazioni nella pipeline.
3. Implementare il metodo `elabora(String input)` che applica in sequenza tutte le funzioni all'input e restituisce il risultato finale.
4. Nel `main`, istanziare la classe e aggiungere le seguenti fasi usando **lambda** o **method references**:
 - Rimozione degli spazi bianchi iniziali e finali (`trim`).
 - Conversione di tutto il testo in maiuscolo.
 - Sostituzione di tutti gli spazi interni con il carattere underscore (`_`).
 - Concatenazione del prefisso "[PROCESSED] " all'inizio della stringa.

Obiettivo: Interfacce Funzionali e Pipeline

Sperimentare la composizione di comportamenti tramite interfacce funzionali standard del package `java.util.function`.

3 Utility per Liste Generiche

Implementare una classe **ListUtility** contenente i seguenti metodi statici generici:

1. **filtra**: riceve in input una lista di tipo `T` e un `Predicate<T>`. Restituisce una nuova lista contenente solo gli elementi che soddisfano il predicato.
2. **trovaMinimo**: riceve in input una lista di elementi di tipo generico `T` e ne restituisce il valore minimo.

Attenzione: Vincoli sui Generics

Per il metodo `trovaMinimo`, è obbligatorio utilizzare il costrutto `<T extends Comparable<T>` per garantire che gli elementi siano confrontabili tra loro.

4 Gestione Log Eventi

Si vuole gestire un sistema di log cronologico.

1. Si definisca la classe **LogEntry** con un **timestamp** (`LocalDateTime`) e un **messaggio** (`String`).
2. Implementare l'interfaccia `Comparable` per ordinare i log in base al tempo (dal più recente al meno recente).
3. Si crei una classe **RegistroEventi** che utilizza un `TreeSet` per memorizzare i log.
4. Implementare un metodo `eseguiAzioneSuLog(Consumer<LogEntry> action)` che applica l'azione fornita a ogni log nel registro.

💡 Tip: Java Time API

Per gestire il tempo in Java moderno, utilizzate `java.time.LocalDateTime`. Per l'ordinamento decrescente, prestate attenzione all'ordine degli operandi nel metodo `compareTo`.